Git en el Servidor - Los Protocolos

1. Los Protocolos de Transferencia Para colaborar en Git, necesitas un repositorio intermedio (remoto). Git utiliza cuatro protocolos principales para mover los datos:

- Protocolo Local: Utiliza carpetas compartidas en un mismo disco o red. Es fácil de configurar aprovechando los permisos existentes, pero suele ser lento y difícil de acceder desde el exterior.
- Protocolos HTTP/HTTPS: Se dividen en dos. El HTTP "inteligente" es hoy el más popular: funciona por los puertos estándar web (atravesando cortafuegos), cifra los datos y solo pide contraseña cuando es estrictamente necesario. El HTTP "tonto" es una versión antigua que solo sirve para lectura.
- Protocolo SSH: Es el estándar para servidores privados. Es muy seguro, rápido y los datos viajan encriptados. Su única desventaja es que requiere que todos los usuarios estén autenticados (mediante claves), por lo que no permite accesos públicos o anónimos.
- Protocolo nativo de Git: Es extremadamente rápido e ideal para proyectos públicos masivos. Sin embargo, carece de cualquier mecanismo de seguridad o autenticación, por lo que cualquiera podría acceder a él.

2. Configuración Básica del Servidor Para poner tu código en un servidor, primero debes crear una versión "básica" o vacía de tu repositorio. Este tipo de repositorio solo contiene la base de datos de Git y el historial, pero no contiene los archivos de trabajo editables. Una vez creado, simplemente lo copias a tu servidor y, si tu equipo tiene acceso por SSH, ya pueden empezar a colaborar inmediatamente.

3. Gestión de Claves y Accesos En entornos corporativos o privados, la autenticación suele hacerse mediante claves públicas SSH. Cada desarrollador genera una clave en su computadora y se la envía al administrador. El administrador guarda estas claves en el servidor bajo un único usuario (por ejemplo, un usuario llamado "git"). Para garantizar la seguridad del servidor, el administrador restringe a este usuario para que única y exclusivamente pueda procesar transferencias de código, impidiendo que nadie navegue por el sistema operativo.

4. Interfaces Web (GitWeb y GitLab) Trabajar solo desde la terminal puede ser limitante, por lo que es común instalar interfaces gráficas web:

- GitWeb: Es una herramienta muy básica y ligera que viene incluida con Git, ideal para visualizar el historial y los archivos del proyecto desde el navegador.
- GitLab: Es una plataforma moderna, robusta y de código abierto. Permite una gestión avanzada y visual: creación de cuentas de usuario, asignación de permisos por grupos, configuración de proyectos (privados, internos o públicos) y colaboración controlada a través de "peticiones de fusión" (donde el código se revisa y discute antes de ser aprobado).

5. Alojamiento Externo Si un equipo no desea invertir tiempo y recursos en configurar, asegurar y mantener su propio servidor físico o virtual, la mejor alternativa es utilizar servicios de alojamiento externo en la nube. Plataformas de terceros se encargan de toda

la infraestructura técnica, lo cual es especialmente ventajoso para visibilizar proyectos de código abierto y facilitar la participación de la comunidad global.